

# PRIVATE ( - )

Modificador private ( Encapsulamiento )

Estamos diseñando el sistema de la ventanilla ciudadana y tenemos la clase **Ciudadano**. Queremos guardar datos muy sensibles, como su **nombre**, **teléfono** o **CURP**. Si dejamos estos datos sin protección, cualquier otra parte del programa podría modificarlos por accidente y arruinar el trámite.



Para aplicar un **encapsulamiento** puro en esta etapa inicial, los atributos deben ser declarados como **private**. Esto obliga a proteger los datos, y si alguien quiere modificarlos o leerlos, tendrá que hacerlo únicamente a través de métodos controlados, conocidos como **getters** y **setters**.

```
public class Ciudadano {
    // Nadie fuera de esta clase puede tocar este dato directamente
    private String curp;

    // Solo a través de este método "getter" público pueden leerlo
    public String getCurp() {
        return curp;
    }

    // Método "setter" público para modificar el dato de forma segura
    public void setCurp(String curp) {
        this.curp = curp;
    }
}
```

# PROTECTED ( # )

Modificador protegido ( La clave para la herencia )

Estamos programando la nómina de la empresa y creamos una clase padre **Empleado** que tiene el atributo **salarioBase**. Luego, creamos clases hijas como **Gerente** o **Vendedor** que necesitan acceder directamente a ese salario para sumarle un bono o su comisión de ventas para calcular su pago final. Si el salario fuera **private**, las clases hijas no podrían tocarlo de forma directa y sería frustrante.



Dentro del tema de herencia, el modificador más amigable será **protected**. Utilizar **protected** en los atributos de las clases padre permite que las clases hijas accedan directamente a los datos, evitando llamadas complejas y facilitando el uso del **polimorfismo**.

```
public class Empleado {
    // La clase hija "Gerente" puede usar esta variable sin restricciones
    protected double salarioBase;
}

public class Gerente extends Empleado {
    public double calcularPago() {
        // Accede a salarioBase directamente gracias a "protected"
        return salarioBase + 5000.0;
    }
}
```

# PUBLIC ( + )

Modificador de libre acceso

Ya tenemos nuestras clases seguras y listas. Ahora, en nuestra clase principal (**Main**), queremos ejecutar nuestro programa crear un ciudadano y procesar su solicitud de licencia. Necesitamos un puente de **comunicación libre de restricciones** para dar la orden de arranque.



Todos los **métodos** y los **constructores** con los que necesitemos interactuar directamente (como **procesarSolicitud()** o **mostrarInformacion()**) deben ser declarados como **public** para que puedan ser invocados sin problemas desde la clase **Main** al momento de hacer las pruebas. Véanlos como las "ventanillas de atención" que permiten interactuar con el objeto de forma segura.

```
public class SolicitudLicencia {
    private boolean estaAprobada; // Dato protegido

    public SolicitudLicencia (boolean estaAprobada) {
        this.estaAprobada = False;
    }

    // Método público que cualquiera puede llamar para procesar el trámite
    public void procesarSolicitud() {
        this.estaAprobada = true;
        System.out.println("Solicitud aprobada.");
    }
}
```

# DEFAULT

Package-Private

Imaginen que están agrupando todas las clases relacionadas con la verificación de documentos en una sola carpeta (llamada **paquete** en Java). Tienen una clase llamada **LectorHuellas** que ayuda a validar la identidad. **Esta clase solo debería ser usada por otras clases dentro de ese mismo paquete**, pero no queremos que el sistema de cobros o el sistema de citas tengan acceso a ella.



Simplemente no se escribe ningún modificador. Cuando omiten escribir **public**, **private** o **protected**, Java aplica automáticamente el acceso por defecto (**default**). Esto significa que solo las clases que "viven en el mismo paquete" podrán interactuar con ella.

```
class LectorHuellas {
    void escanear() {
        System.out.println("Escaneando huella...");
    }
}
```